

Score-Based Generative Model

Jiahao Song

Department of Mathematics and Statistics
Shanxi University

September 2024

Table of Contents

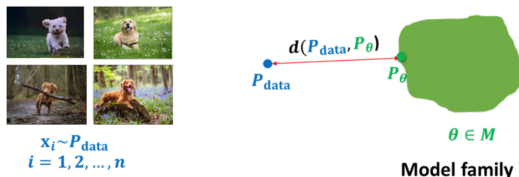
- 1 Introduction
- 2 The score function, score-based models, and score matching
- 3 Langevin dynamics
- 4 Multi-scale noise perturbation to improve score estimation
- 5 Score-based generative modeling with SDEs

Table of Contents

- 1 Introduction
- 2 The score function, score-based models, and score matching
- 3 Langevin dynamics
- 4 Multi-scale noise perturbation to improve score estimation
- 5 Score-based generative modeling with SDEs

What are the objectives of generative model?

The overall objectives of generative model can be summarized as follows:



- Given observed samples $\mathbf{x}$ from a dataset we are interested, the goal of a **generative model** is to learn to model its true data distribution $p_{\text{data}}(\mathbf{x})$.
- Once learned, we can generate new samples from our approximate model $p_{\theta}(\mathbf{x})$.
- Under some formulations, we are able to use the learned model $p_{\theta}(\mathbf{x})$ to evaluate the likelihood of observed or sampled data.

Current generative Models

Existing generative modeling techniques can largely be grouped into two categories based on how they represent probability distributions.

- **Likelihood-based models:** learn the distribution's probability density (or mass) function via (approximate) maximum likelihood.
 - **Typical models:** GMM, Autoregressive models, VAEs, Normalize flow models and Energy-based models.
 - **Pros:** Principled model comparison via likelihoods.
 - **Cons:** Hard to deal with normalizing constant for likelihood computation, or must rely on surrogate objectives to approximate maximum likelihood training.
- **Adversarial learning models:** two competing models improve each other through a game-like interaction, leading to better generative performance.
 - **Typical models:** GANs.
 - **Pros:** Samples typically have better quality.
 - **Cons:** Training instability, mode collapse, and unable to represent probability distribution.

Score-based Generative Model

Beyond maximum likelihood and adversarial learning, **Score-based Generative Model** is proposed to deal with these limitations.

The key idea is to model the gradient of the log probability density function $\nabla \log \mathbf{p}_{\text{data}}(\mathbf{x})$, a quantity often known as the **score function**

Such score-based models are not required to have a tractable normalizing constant, and can be directly learned by **score matching**.

Wide range of applications

Score-based models have achieved state-of-the-art performance on many downstream tasks and applications.

- **Image generation**
- **Image inpainting, image colorization, medical image reconstruction**
- **Audio synthesis**
- **Music generation**

Table of Contents

- 1 Introduction
- 2 The score function, score-based models, and score matching
- 3 Langevin dynamics
- 4 Multi-scale noise perturbation to improve score estimation
- 5 Score-based generative modeling with SDEs

How to represent probability distributions?

Suppose we are given a dataset $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$, where each point is drawn independently from an underlying data distribution $p_{data}(\mathbf{x})$.

The goal of generative modeling is to fit a model to the data distribution such that we can synthesize new data points at will by sampling from the distribution.

In order to build such a generative model, we first need a way to **represent a probability distribution**. One such way, as in **likelihood-based models**, is to directly model the **probability density function** (p.d.f.) or **probability mass function** (p.m.f.).

Example: Energy-based Models

Let $f_\theta(\mathbf{x}) \in \mathbb{R}$ be a real-valued function parameterized by a learnable parameter θ . We can define a p.d.f. via

$$p_\theta(\mathbf{x}) = \frac{e^{f_\theta(\mathbf{x})}}{Z_\theta} \quad (1)$$

where $Z_\theta > 0$ is a **normalizing constant** dependent on θ , such that

$$\int_{\mathbf{x}} p_\theta(\mathbf{x}) d\mathbf{x} = 1 \quad (2)$$

Here the function $f_\theta(\mathbf{x})$ is often called an unnormalized probabilistic model, or **energy-based model**. We can train $p_\theta(\mathbf{x})$ by **maximizing the log-likelihood** of the data

$$\max_{\theta} \sum_{i=1}^N \log p_\theta(\mathbf{x}_i) \quad (3)$$

Normalizing constant Z_θ is intractable

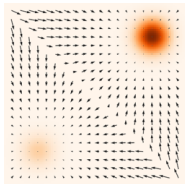
We must evaluate the normalizing constant Z_θ to compute $p_\theta(\mathbf{x})$. However it's a typically **intractable** quantity for any general $f_\theta(\mathbf{x})$

Thus to make maximum likelihood training feasible, likelihood-based models must:

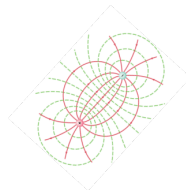
- **restrict their model architectures**
 - causal convolutions in autoregressive models
 - invertible networks in normalizing flow models
- **approximate the normalizing constant**
 - variational inference in VAEs
 - MCMC sampling used in contrastive divergence

Estimating score function

In order to solve this problem, instead of modeling data distribution $p_{data}(\mathbf{x})$, we can model the **score function** $\nabla_{\mathbf{x}} \log p_{data}(\mathbf{x})$, when the p.d.f. is differentiable.



pdf and score



electrical potentials and fields

scores and probability densities are also equivalent ways of describing the same continuous probability distribution.

Estimating score function

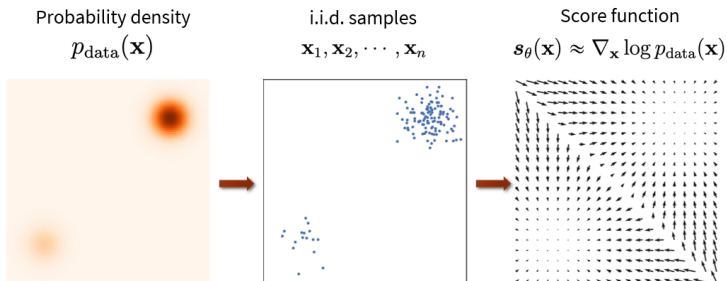
If we consider energy-based models, according to equation (1), we can model the score function as follow:

$$s_{\theta}(\mathbf{x}) = \nabla_{\mathbf{x}} \log p_{\theta}(\mathbf{x}) = \nabla_{\mathbf{x}} \log f_{\theta}(\mathbf{x}) - \nabla_{\mathbf{x}} \log Z_{\theta} = \nabla_{\mathbf{x}} \log f_{\theta}(\mathbf{x}) \quad (4)$$

- $s_{\theta}(\mathbf{x})$ is often called a **score-based model**.
- learning $s_{\theta}(\mathbf{x})$ can avoid estimating normalizing constant Z_{θ} .
- f_{θ} is usually defined using a **neural network**, so we can calculate its gradients efficiently by **backpropagation algorithm**.
- The score-based model is learned to estimate ground truth score function, i.e. $s_{\theta}(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p_{data}(\mathbf{x})$.

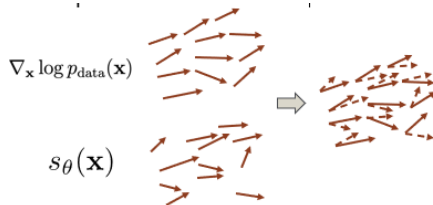
Estimating score function

By training score-based model $s_\theta(\mathbf{x})$, we can estimate ground truth score function $\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$



How to train score-based model?

- **Given:** i.i.d. samples $\{\mathbf{x}_1, \mathbf{x}_1, \dots, \mathbf{x}_N\} \sim p_{data}(\mathbf{x})$.
- **Task:** Estimating the ground truth score function $\nabla_{\mathbf{x}} \log p_{data}(\mathbf{x})$.
- **Score-based Model:** A learnable vector-valued function $s_{\theta}(\mathbf{x}) : \mathbb{R}^d \rightarrow \mathbb{R}^d$.
- **Goal:** $s_{\theta}(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p_{data}(\mathbf{x})$.



Intuitively, We can use **Average Euclidean Distance** over the space.

How to train score-based model?

The average euclidean distance between gredients of probability densities is also called **Fisher Divergence**.

Objective: Fisher Divergence

In score-based model, the training objective function is actually Fisher Divergence:

$$\frac{1}{2} \mathbb{E}_{\mathbf{x} \sim p_{data}} [\|\nabla_{\mathbf{x}} \log p_{data}(\mathbf{x}) - s_{\theta}(\mathbf{x})\|_2^2] \quad (5)$$

- Fisher Divergence measures the difference between two distributions through the gradient of the density function. It is particularly effective in **high-dimensional data** and **manifold learning**.
- In score-based model, by minimizing the objective (5), we are effectively minimizing the difference between $p_{data}(\mathbf{x})$ and $p_{\theta}(\mathbf{x})$ through Fisher Divergence.

Score Matching

However, objective (5) is not computable, because we don't know the ground truth data distribution $p_{data}(\mathbf{x})$. **Score matching** is proposed to deal with the problem.

Score Matching (Hyvärinen 2005)¹

We can train the score-based model by score matching:

$$\mathbb{E}_{\mathbf{x} \sim p_{data}} \left[\frac{1}{2} \|s_{\theta}(\mathbf{x})\|_2^2 + \text{tr}(\nabla_{\mathbf{x}} s_{\theta}(\mathbf{x})) \right] \quad (6)$$

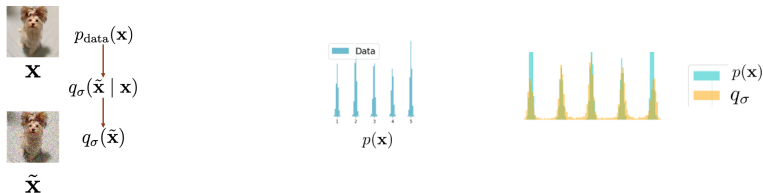
- $\nabla_{\mathbf{x}} s_{\theta}(\mathbf{x}) = \nabla_{\mathbf{x}}^2 \log p_{\theta}(\mathbf{x})$, so $\text{tr}(\nabla_{\mathbf{x}} s_{\theta}(\mathbf{x}))$ is actually **Hessian Matrix's** trace of $\log p_{\theta}(\mathbf{x})$.
- compute $\text{tr}(\nabla_{\mathbf{x}} s_{\theta}(\mathbf{x}))$ is expensive, making score matching not scalable.

¹ Hyvärinen A, Dayan P. Estimation of non-normalized statistical models by score matching[J]. Journal of Machine Learning Research, 2005, 6(4).

Denoising Score Matching

Estimating score function for ground truth distribution $p_{data}(\mathbf{x})$ is difficult, we can consider the noise perturbed distribution:

$$q_{\sigma}(\tilde{\mathbf{x}}|\mathbf{x}) = \mathcal{N}(\mathbf{x}; \sigma^2 \mathbf{I}), \quad q_{\sigma}(\tilde{\mathbf{x}}) = \int p_{data}(\mathbf{x}) q_{\sigma}(\tilde{\mathbf{x}}|\mathbf{x}) d\mathbf{x} \quad (7)$$



- score estimation for $\nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}})$ is easier compared to $\nabla_{\mathbf{x}} p_{data}(\mathbf{x})$
- If the noise level σ is small enough, this is a suitable approximation, i.e. $q_{\sigma}(\tilde{\mathbf{x}}) \approx p_{data}(\mathbf{x})$.

Denoising Score Matching

Denoising Score Matching (Vincent 2011)²

We can train score-based model using noise perturbed data:

$$\frac{1}{2} \mathbb{E}_{\tilde{\mathbf{x}} \sim q_{\sigma}} [\|\nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}}) - s_{\theta}(\tilde{\mathbf{x}})\|_2^2] \quad (8)$$

By some derivation, equation (8) is equivalent to:

$$\frac{1}{2} \mathbb{E}_{\mathbf{x} \sim p_{data}(\mathbf{x}), \tilde{\mathbf{x}} \sim q_{\sigma}(\tilde{\mathbf{x}}|\mathbf{x})} [\|s_{\theta}(\tilde{\mathbf{x}}) - \nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}}|\mathbf{x})\|_2^2] \quad (9)$$

² Vincent P. A connection between score matching and denoising autoencoders[J]. Neural computation, 2011, 23(7): 1661-1674.

Denoising Score Matching

The overall process for estimating score function by denoising score matching is as follows:

- Sample a minibatch of datapoints $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\} \sim p_{data}(\mathbf{x})$
- Add noise to get perturbed datapoints $\{\tilde{\mathbf{x}}_1, \tilde{\mathbf{x}}_2, \dots, \tilde{\mathbf{x}}_n\} \sim q_\sigma(\tilde{\mathbf{x}})$
- Estimate the score function by minimize denoising score matching objective function((8),(9)):

$$\frac{1}{2n} \sum_{i=1}^n [\|s_\theta(\tilde{\mathbf{x}}_i) - \nabla_{\tilde{\mathbf{x}}_i} \log q_\sigma(\tilde{\mathbf{x}}_i|\mathbf{x}_i)\|_2^2] \quad (10)$$

if Gaussian perturbation, equation (10) can be written as:

$$\frac{1}{2n} \sum_{i=1}^n \left[\left\| s_\theta(\tilde{\mathbf{x}}_i) + \frac{\tilde{\mathbf{x}}_i - \mathbf{x}_i}{\sigma^2} \right\|_2^2 \right] \quad (11)$$

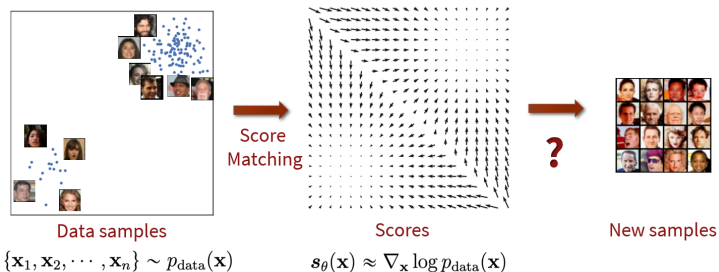
- Stochastic gradient descent until convergence.

Table of Contents

- 1 Introduction
- 2 The score function, score-based models, and score matching
- 3 Langevin dynamics**
- 4 Multi-scale noise perturbation to improve score estimation
- 5 Score-based generative modeling with SDEs

How to sample using scores?

Once we have trained a score-based model $s_\theta(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$, the next problem we should consider is sampling.



We can use an iterative procedure called **Langevin dynamics** to draw samples from it.

Langevin dynamics sampling

Langevin dynamics (Parisi 1981)³

Langevin dynamics provides an MCMC procedure to sample from a distribution $p(\mathbf{x})$ using only its score function $\nabla_{\mathbf{x}} \log p(\mathbf{x})$.

- Initialize $\mathbf{x}^0 \sim \pi(\mathbf{x})$
 - Repeat for $t \leftarrow 1, 2, \dots, T$:
 - $\mathbf{z}^t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - $\mathbf{x}^t \leftarrow \mathbf{x}^{t-1} + \frac{\epsilon}{2} \nabla_{\mathbf{x}} \log p(\mathbf{x}^{t-1}) + \sqrt{\epsilon} \mathbf{z}^t$
 - if $\epsilon \rightarrow 0$ and $T \rightarrow \infty$, we are guaranteed to have $\mathbf{x}^T \sim p(\mathbf{x})$
-
- By trained a score-based model $s_{\theta}(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p_{data}(\mathbf{x})$, we can sample new samples using Langevin dynamics.
 - π is prior distribution, usually set to Gaussian distribution.

³ Parisi G. Correlation functions and computer simulations[J]. Nuclear Physics B, 1981, 180(3): 378-384.

Langevin dynamics sampling

Through combining score matching and Langevin dynamics, we can generate new samples from the data distribution $p_{data}(\mathbf{x})$.

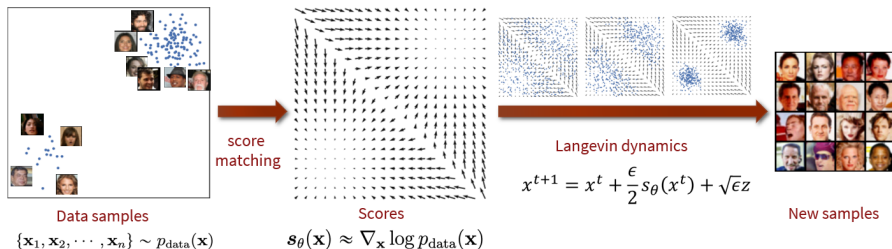


Table of Contents

- 1 Introduction
- 2 The score function, score-based models, and score matching
- 3 Langevin dynamics
- 4 Multi-scale noise perturbation to improve score estimation
- 5 Score-based generative modeling with SDEs

Pitfalls of current method

So far, we've discussed how to train a score-based model with score matching, and then produce samples via Langevin dynamics.

However, this naive approach has had limited success in practice—we'll talk about some pitfalls of score matching(Song 2019)⁴:

- **Manifold hypothesis**: data score is undefined in some data space.
- **Low data density**: Score matching fails in low data density regions.

⁴ Song Y, Ermon S. Generative modeling by estimating gradients of the data distribution[J]. Advances in neural information processing systems, 2019, 32.

Manifold hypothesis

The manifold hypothesis states that data in the real world tend to concentrate on low dimensional manifolds embedded in a high dimensional space (a.k.a., the ambient space). This hypothesis has become the foundation of manifold learning (Roweis 2000)⁵.



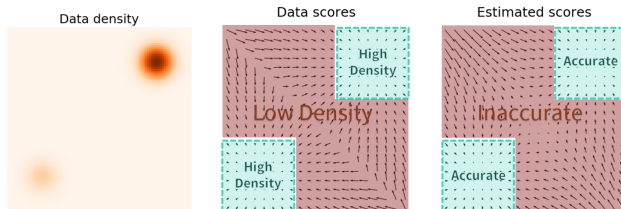
With manifold hypothesis, current method will face two key difficulties:

- First, the score $\nabla_{\mathbf{x}} \log p_{data}(\mathbf{x})$ is taken in the ambient space, it is undefined when $\mathbf{x}$ is confined to a low dimensional manifold.
- Second, the score matching objective provides a consistent score estimator only when the support is the whole space.

⁵ Roweis S T. Nonlinear dimensionality reduction by locally linear embedding[J]. Science, 2000, 290: 232.

Low data density

Besides Manifold hypothesis, the scarcity of data in low density regions can cause difficulties for both score estimation with score matching and MCMC sampling with Langevin dynamics.

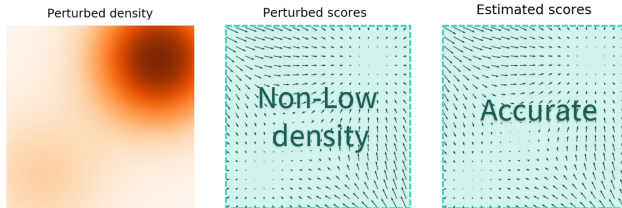


- The estimated score functions are inaccurate in low density regions, where few data points are available for training.
- When sampling with Langevin dynamics, the initial sample $\mathbf{x}^0$ is highly likely in low density regions.
- Having an inaccurate score estimation will derail Langevin dynamics from the very beginning of the procedure, preventing it from generating high quality samples.

Solutions

In order to solve these pitfalls, we can perturb data points with Gaussian noise. The reasons are as follows:

- First, since the support of our Gaussian noise distribution is the whole space, the perturbed data will not be confined to a low dimensional manifold, which obviates difficulties from the manifold hypothesis and makes score estimation well-defined.
- Second, large Gaussian noise has the effect of filling low density regions in the original unperturbed data distribution; therefore score matching may get more training signal to improve score estimation.



There is another question remains: how to choose an appropriate noise scale for the perturbation?

- **Larger noise** can obviously cover more low density regions for better score estimation, but it over-corrupts the data and alters it significantly from the original distribution.
- **Smaller noise**, on the other hand, causes less corruption of the original data distribution, but does not cover the low density regions.

To achieve the best of both worlds, we can use **multiple scales of noise perturbation** and **noise conditional score network** to simultaneously estimate scores corresponding to all noise levels.(Song 2019)⁴

⁴ Song Y, Ermon S. Generative modeling by estimating gradients of the data distribution[J]. Advances in neural information processing systems, 2019, 32.

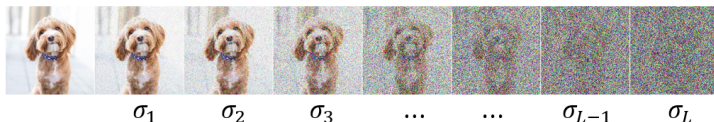
Multi-scale noise perturbation

Suppose we perturb the data with isotropic Gaussian noise, and let there be a total of L increasing standard deviations $\sigma_1 < \sigma_2 < \dots < \sigma_L$, such that σ_L is large enough to mitigate the difficulties we discussed above.

we first perturb the data distribution $p_{data}(\mathbf{x})$ with each of Gaussian noise $\mathcal{N}(\mathbf{0}, \sigma_i^2 \mathbf{I})$, $i = 1, 2, \dots, L$ to obtain a sequence of noise-perturbed distributions:

$$p_{\sigma_i}(\mathbf{x}) = \int p_{data}(\mathbf{y}) \mathcal{N}(\mathbf{x}; \mathbf{y}, \sigma_i^2 \mathbf{I}) d\mathbf{y}, \quad i = 1, 2, \dots, L$$

Note that we can easily draw samples from $p_{\sigma_i}(\mathbf{x})$ by sampling $\mathbf{x} \sim p_{data}(\mathbf{x})$ and computing $\mathbf{x} + \sigma_i \mathbf{z}$ with $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.



Noise Conditional Score Network

Next, we estimate the score function of each noise-perturbed distribution, $\nabla_{\mathbf{x}} \log p_{\sigma_i}(\mathbf{x})$, by training a **Noise Conditional Score Network** $s_{\theta}(\mathbf{x}, \sigma_i)$ (NCSN) with score matching, such that $s_{\theta}(\mathbf{x}, \sigma_i) \approx \nabla_{\mathbf{x}} \log p_{\sigma_i}(\mathbf{x})$ for all $i = 1, 2, \dots, L$.

According to denoising score matching (11), the training objective for NCSN can be written as follows:

$$\mathcal{L}(\theta; \{\sigma_i\}_{i=1}^L) \triangleq \frac{1}{L} \sum_{i=1}^L \lambda(\sigma_i) \ell(\theta; \sigma_i) \quad (12)$$

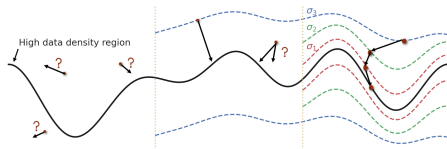
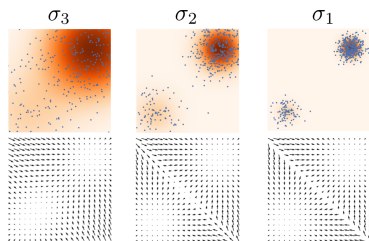
$$\ell(\theta; \sigma_i) = \frac{1}{2} \mathbb{E}_{\mathbf{x} \sim p_{data}(\mathbf{x}), \tilde{\mathbf{x}} \sim \mathcal{N}(\mathbf{x}, \sigma_i^2 \mathbf{I})} \left[\left\| s_{\theta}(\tilde{\mathbf{x}}, \sigma_i) + \frac{\tilde{\mathbf{x}} - \mathbf{x}}{\sigma_i^2} \right\|_2^2 \right] \quad (13)$$

- $\lambda(i) \in \mathbb{R} > 0$ is a positive weighting function.

How to sample?

Same question as above, how to sample after we train a NCSN?

We can get some intuitions from a few images below:



Under these intuitions, we can use **annealed Langevin dynamics** to generate new samples using NCSN.

Annealed Langevin Dynamics

Algorithm 1: Annealed Langevin dynamics (Song 2019)⁴

```
1: Input:  $\{\sigma_i\}_{i=1}^L, \epsilon, T, \pi, s_\theta$ 
2: Output: generated sample  $\tilde{\mathbf{x}}^0$ 
3: Initialize  $\mathbf{x}^0 \sim \pi$ 
4: for  $i \leftarrow 1$  to  $L$  do
5:    $\alpha_i \leftarrow \epsilon \cdot \sigma_i^2 / \sigma_L^2$ 
6:   for  $t \leftarrow 1$  to  $T$  do
7:     Draw  $\mathbf{z}^t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
8:      $\mathbf{x}^t \leftarrow \mathbf{x}^{t-1} + \frac{\alpha_i}{2} s_\theta(\mathbf{x}^{t-1}, \sigma_i) + \sqrt{\alpha_i} \mathbf{z}^t$ 
9:   end for
10:   $\mathbf{x}^0 \leftarrow \mathbf{x}^T$ 
11: end for
12:  $\tilde{\mathbf{x}}^0 \leftarrow \mathbf{x}^T$ 
13: return  $\tilde{\mathbf{x}}^0 = 0$ 
```

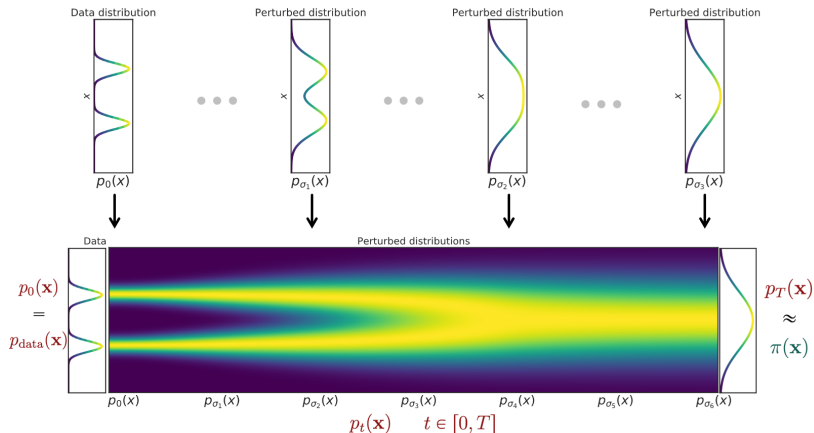
⁴ Song Y, Ermon S. Generative modeling by estimating gradients of the data distribution[J]. Advances in neural information processing systems, 2019, 32.

Table of Contents

- 1 Introduction
- 2 The score function, score-based models, and score matching
- 3 Langevin dynamics
- 4 Multi-scale noise perturbation to improve score estimation
- 5 Score-based generative modeling with SDEs

Infinite noise levels

As we already discussed, adding multiple noise scales is critical to the success of score-based generative models. By generalizing the number of noise scales to infinity, we can obtain higher quality samples.



Forward noising SDEs

When the number of noise scales approaches infinity, we essentially perturb the data distribution with continuously growing levels of noise. In this case, the noise perturbation procedure is a **continuous-time stochastic process**

We can use a SDE to define the multi-scale noise perturbation process:

$$d\mathbf{x} = f(\mathbf{x}, t)dt + g(t)d\mathbf{w} \quad (14)$$

- $f(\cdot, t) : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is a vector-valued function called the **drift coefficient**.
- $g(t) \in \mathbb{R}$ is a real-valued function called the **diffusion coefficient**.
- $\mathbf{w}$ denotes a standard **Wiener process (a.k.a. Brownian motion)**, and $d\mathbf{w}$ can be viewed as infinitesimal white noise.

Reversed denoising SDEs

Recall that with a finite number of noise scales, we can generate samples by reversing the perturbation process with annealed Langevin dynamics, i.e., sequentially sampling from each noise-perturbed distribution using Langevin dynamics.

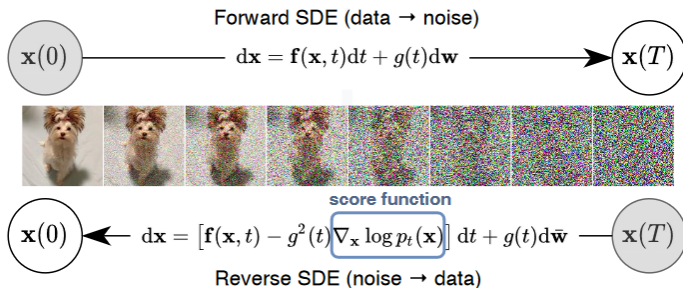
For infinite noise scales, we can analogously reverse the perturbation process (14) for sample generation by using the reverse SDE:

$$d\mathbf{x} = [f(\mathbf{x}, t) - g^2(t)\nabla_{\mathbf{x}} \log p_t(\mathbf{x})] dt + g(t)d\mathbf{w} \quad (15)$$

- $p_t(\mathbf{x})$ denote the marginal probability density of random variable at time t .
- $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$ is the score function, we can use score-based model $s_{\theta}(\cdot, t)$ to estimate it.

Score-based Generative Model via SDEs

To summarize, the entire addition and denoising process is shown in the following figure:



SDE to ODE: Deterministic Trajectories

Stochastic Differential Equations (SDEs) provide a **random trajectory** for both the forward diffusion (adding noise) and the reverse diffusion (denoising) processes.

If we desire a **deterministic trajectory** instead of a random one, we can use an Ordinary Differential Equation (ODE) to describe the process.

The ODE corresponding to reverse SDE (15) can be written as:

$$\frac{d\mathbf{x}}{dt} = f(\mathbf{x}, t) - \frac{1}{2}g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \quad (16)$$

- We can prove that the marginal distribution induced by ODE Eq. (16) is same with that of SDE Eq. (15).
- The ODE can be seen as the **expected trajectory** of the SDE.
- The ODE (16) is named as **probability flow ODE**⁵

⁵ Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, Ben Poole. "Score-Based Generative Modeling through Stochastic Differential Equations." International Conference on Learning Representations. 2021.

Probability flow ODE

By using ODE instead of SDE, we can achieve the following benefits:

- **More Efficient Sampling:** ODEs provide a deterministic path, which reduces the number of steps required for sampling, resulting in faster generation compared to SDEs.
- **More Precise Likelihood Computation:** Leveraging the connection to **neural ODEs**⁶, we can compute the density defined by ODE via the instantaneous change of variables formula:

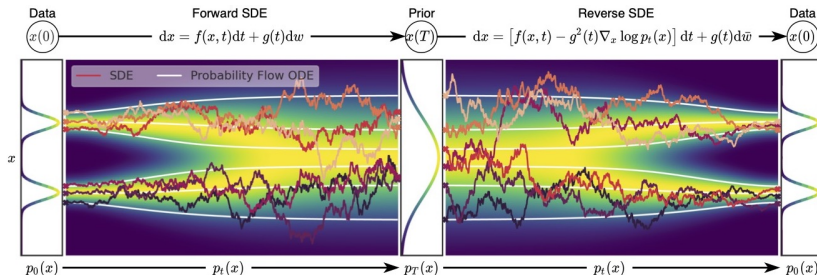
$$\log p_0(\mathbf{x}) = \log p_T(\mathbf{x}) - \int_0^T \nabla \cdot [f(\mathbf{x}, t) - g^2(t) \nabla_{\mathbf{x}} \log p_t(\mathbf{x})] dt \quad (17)$$

This formula allows for exact likelihood computation, which is crucial for evaluating generative models.

⁶ Chen, Ricky TQ, Yulia, Rubanova, Jesse, Bettencourt, David K, Duvenaud. "Neural ordinary differential equations". Advances in neural information processing systems 31. (2018).

Overview of score-based generative modeling through SDEs

We can use the following figure to summarize:



- We can map data $x(0)$ to a noise distribution $x(T)$ (the prior) with an SDE, and reverse this SDE for sampling.
- We can also reverse the associated probability flow ODE, which yields a deterministic process that samples from the same distribution as the SDE.
- Both the reverse-time SDE and probability flow ODE can be obtained by estimating the score $\nabla_x \log p_t(x)$.

Discretization of SDEs

After establishing the general theoretical framework, we need to discretize the SDE using numerical methods for practical computation. The simplest method for discretizing an SDE is the **Euler-Maruyama method**.

For a general forward SDE (14) and reverse SDE (15), the discretized difference equations are as follows:

Forward Difference Equation:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + f(\mathbf{x}_t, t)\Delta t + g(t)\Delta \mathbf{w}_t \quad (18)$$

Reverse Difference Equation:

$$\mathbf{x}_{t-1} = \mathbf{x}_t + [f(\mathbf{x}_t, t) - g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x})] \Delta t + g(t)\Delta \mathbf{w}_t \quad (19)$$

By modifying the drift term $f(\mathbf{x}, t)$ and the diffusion term $g(t)$, we can obtain different equations. Next, we will discuss some common settings.

VP and VE SDE

We introduce two kinds of SDEs, VP and VE SDE⁵, which can be seen as a continuous diffusion process of **NCSN** and **DDPM**⁷:

	Continuous	Discrete
VP	$dx = -\frac{1}{2}\beta(t)xdt + \sqrt{\beta(t)}dw$	$\mathbf{x}_t = \sqrt{1 - \beta(t)}\mathbf{x}_{t-1} + \sqrt{\beta(t)}\mathbf{z}_{t-1}$
VE	$d\mathbf{x} = \sqrt{\frac{d[\sigma^2(t)]}{dt}}d\mathbf{w}$	$\mathbf{x}_t = \mathbf{x}_{t-1} + \sqrt{\sigma^2(t) - \sigma^2(t-1)}\mathbf{z}_{t-1}$

- **VP(Variance Preserving) SDE**: Setting drift term $f(\mathbf{x}, t) = -\frac{1}{2}\beta(t)$, and diffusion term $g(t) = \sqrt{\beta(t)}$ in Eq (14).
 - The variance of the data remains unchanged.
 - Its discrete form corresponds to the diffusion process in DDPM.
- **VE(Variance Exploding) SDE**: Setting drift term $f(\mathbf{x}, t) = 0$, and diffusion term $g(t) = \sqrt{\frac{d\sigma^2(t)}{dt}}$ in Eq (14).
 - The variance of the data increases rapidly.
 - Its discrete form corresponds to the diffusion process in NCSN.

⁷ Ho, Jonathan, Ajay, Jain, Pieter, Abbeel. "Denoising diffusion probabilistic models". Advances in neural information processing systems 33. (2020): 6840–6851.

The overall training process of SGM via SDE can be summarize as follows:

- Sample a minibatch of data points $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\} \sim p_0(\mathbf{x})$.
- We use forward SDE to convert data points to Gaussian prior:

$$d\mathbf{x} = f(\mathbf{x}, t)dt + g(t)d\mathbf{w}$$

- Then, we use score estimator to model the score function:

$$\theta^* = \arg \min_{\theta} \mathbb{E}_t \left\{ \lambda(t) \mathbb{E}_{\mathbf{x}(0)} \mathbb{E}_{\mathbf{x}(t)|\mathbf{x}(0)} \left[\left\| \mathbf{s}_{\theta}(\mathbf{x}(t), t) - \nabla_{\mathbf{x}(t)} \log p_{0t}(\mathbf{x}(t) | \mathbf{x}(0)) \right\|_2^2 \right] \right\}$$

- If conditional generation:

$$\theta^* = \arg \min_{\theta} \mathbb{E}_t \left\{ \lambda(t) \mathbb{E}_{\mathbf{x}(0)} \mathbb{E}_{\mathbf{x}(t)|\mathbf{x}(0)} \left[\left\| \mathbf{s}_{\theta}(\mathbf{x}(t), y, t) - \nabla_{\mathbf{x}(t)} \log p_{0t}(\mathbf{x}(t) | \mathbf{x}(0), y) \right\|_2^2 \right] \right\}$$

- In conditional generation, our data points are $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$, y is the condition information.

- Once we obtain the score estimator $\mathbf{s}_{\theta^*}(\mathbf{x}, t)$, we should first sample noise data $\mathbf{X}(\mathbf{T})$ from prior Gaussian distribution $p_T(\mathbf{x})$.
- Then we can generate new samples in $p_0(\mathbf{x})$ through reverse score function in reverse SDE Eq (15):

$$d\mathbf{x} = [f(\mathbf{x}, t) - g^2(t)\mathbf{s}_{\theta^*}(\mathbf{x}, t)] dt + g(t)d\mathbf{w} \quad (20)$$

- We can also use corresponding ODE Eq (16) for quickly sampling:

$$\frac{d\mathbf{x}}{dt} = f(\mathbf{x}, t) - \frac{1}{2}g^2(t)\mathbf{s}_{\theta^*}(\mathbf{x}, t) \quad (21)$$

- Finally we can get new data $\hat{\mathbf{X}}(\mathbf{0})$.

• Class-Free Guidance

- This technique blends unconditional and conditional generation by introducing a weighting factor.
- Formula:

$$\hat{s}_{\theta}(x_t, t|y) = (1 - w)s_{\theta}(x_t, t|\emptyset) + ws_{\theta}(x_t, t|y)$$

where w is the guidance weight, y is the condition, and $\emptyset$ represents the unconditional input.

• Noise Schedule Optimization

- Optimizing the noise schedule can significantly improve the model's generation quality.
- Common schedules: Linear, cosine.
- Cosine schedule formula:

$$\beta(t) = 1 - \cos\left(\frac{\pi t}{T}\right)$$